

eBPF Programming and sched-ext

Disk I/O accounting + a minimal FIFO scheduler

Two related ideas

eBPF	sched-ext
Sandboxed programs executed by the kernel verifier/JIT	A Linux scheduler class implemented through eBPF callbacks
Attach to events: tracepoints, kprobes, networking, cgroups	Define task placement and dispatch decisions
Great for observation, filtering, and policy	A specialized eBPF application with stronger correctness constraints

The eBPF application model

```
event / hook
  |
  v
BPF program --> BPF map <--> user-space program
  |                               |
  +----- verifier + JIT -----+----- report / configure
```

- **Program:** restricted C compiled to BPF bytecode.
- **Hook:** the kernel event that invokes the program.
- **Map:** shared, typed kernel-resident state.
- **Loader:** user space creates, loads, attaches, and reads the application.

Why the verifier matters

- Verifier proves memory safety, bounded execution, and safe helper access before a program can load.
- Programs cannot freely dereference kernel pointers or call arbitrary kernel functions.
- Helpers and kfuncs are the intentional interface to kernel services.
- JIT compiles accepted bytecode to native instructions on supported architectures.

Example: account disk I/O by process

We will use the following two tracepoints:

1. At `block_rq_issue`, remember the current process for the issued request.
2. At `block_rq_complete`, find that process and add the completed byte count.
3. Prints and clears the per-process counters every second.

Shared types used by BPF and user space

```
struct who_t {  
    __u32 pid;  
    char name[TASK_COMM_LEN];  
};  
  
struct val_t {  
    __u64 bytes;  
};
```

Keep these definitions in a header file so map keys and values have identical layouts on both sides of the boundary.

Maps

```
/* outstanding request -> process that issued it */  
BPF_MAP_TYPE_HASH  whobyreq;  
/* process -> accumulated completed bytes */  
BPF_MAP_TYPE_HASH  counts;
```

Map	Key	Value	Lifetime
whobyreq	struct request *	who_t	issue until completion
counts	who_t	val_t	one reporting interval

Capture attribution at request issue

```
SEC("tp_btf/block_rq_issue")
int BPF_PROG(handle_block_rq_issue, struct request *request)
{
    struct who_t process = {};

    if (bpf_get_current_comm(process.name, sizeof(process.name)))
        return 0;

    process.pid = bpf_get_current_pid_tgid() >> 32;
    bpf_map_update_elem(&whobyreq, &request, &process, BPF_ANY);
    return 0;
}
```

Account at completion

```
SEC("tp_btf/block_rq_complete")
int BPF_PROG(handle_block_rq_complete, struct request *request,
             blk_status_t error, unsigned int bytes)
{
    const struct who_t *process = bpf_map_lookup_elem(&whobyreq, &request);
    struct val_t *value;

    if (!process)
        return 0;

    value = lookup_or_init(&counts, process, &(struct val_t){});
    if (value)
        __sync_fetch_and_add(&value->bytes, bytes);

    bpf_map_delete_elem(&whobyreq, &request);
    return 0;
}
```

User space owns the lifecycle

```
obj = iomem_bpf__open_and_load(); // create maps; verifier validates program
iomem_bpf__attach(obj);           // attach tracepoints

while (!exiting) {
    sleep(1);
    print_and_clear_counts(obj->maps.counts);
}
iomem_bpf__destroy(obj);          // detach and free resources
```

From observing to scheduling

Normal eBPF tracing observes a fixed kernel hook.

sched-ext lets a BPF program implement `struct sched_ext_ops`:

```
while (task in SCHED_EXT)
    if (task can migrate) ops.select_cpu();
    ops.runnable();
    while (task_is_runnable(task))
        if (task is not in a DSQ || task->scx.slice == 0)
            ops.enqueue();
            if (sched_change(task))
                ops.dequeue(); ops.quiescent(); ops.runnable(); continue;
            ops.dispatch(); ops.dequeue();
    ops.running();
    while (task_is_runnable(task) && task->scx.slice > 0)
        ops.tick();
        if (task->scx.slice == 0)
            ops.dispatch();
    ops.stopping();
    ops.quiescent();
```

Minimal FIFO scheduler

Adapted from `scx_simple`, with weighted virtual time and statistics removed:

```
#include "vmlinux.h"
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>

SEC("struct_ops/fifo_select_cpu")
s32 BPF_PROG(fifo_select_cpu, struct task_struct *p,
             s32 prev_cpu, u64 wake_flags)
{
    bool is_idle = false;
    s32 cpu = scx_bpf_select_cpu_dfl(p, prev_cpu, wake_flags, &is_idle);
    if (is_idle)
        scx_bpf_dsq_insert(p, SCX_DSQ_LOCAL, SCX_SLICE_DFL, 0);
    return cpu;
}
```

FIFO enqueue and dispatch

```
SEC("struct_ops/fifo_enqueue")
void BPF_PROG(fifo_enqueue, struct task_struct *p, u64 enq_flags)
{
    scx_bpf_dsq_insert(p, SCX_DSQ_GLOBAL, SCX_SLICE_DFL, enq_flags);
}
```

Register the operations

```
SEC(".struct_ops")
struct sched_ext_ops fifo_ops = {
    .select_cpu = (void *)fifo_select_cpu,
    .enqueue     = (void *)fifo_enqueue,
    .name        = "fifo",
};
```

Enable eBPF and sched-ext

Install the build tools, then verify the **booted kernel**:

```
sudo apt update
sudo apt install -y build-essential clang llvm libbpf-dev bpftool pahole \
    linux-headers-$(uname -r) linux-tools-common linux-tools-$(uname -r)

test -r /sys/kernel/btf/vmlinux && echo "BTF available"
bpftool feature probe kernel | less
grep -E 'CONFIG_(BPF|BPF_SYSCALL|BPF_JIT|DEBUG_INFO_BTF|SCHED_CLASS_EXT)=' /boot/config-$(uname -r)
test -e /sys/kernel/sched_ext/state && cat /sys/kernel/sched_ext/state
```

Build

```
# Generate kernel types from BTF
bpftool btf dump file /sys/kernel/btf/vmlinux format c > vmlinux.h

# Compile the eBPF program
clang -g -O2 -target bpf -D__TARGET_ARCH_x86 -I. \
    -c program.bpf.c -o program.bpf.o

# Generate the libbpf skeleton
bpftool gen skeleton program.bpf.o > program.skel.h

# Compile the user-space loader
cc -g -O2 -Wall -Wextra program.c -lbpf -o program
```

Demo

```
# Example 1: completed block I/O bytes by process
cd disk-io-by-process
make
sudo ./iomem

# Example 2: activate the FIFO scheduler
cd ../sched-ext-fifo
make
sudo ./fifo

# Inspect sched-ext from another terminal
cat /sys/kernel/sched_ext/state
cat /sys/kernel/sched_ext/root/ops
```

Sources

- Linux eBPF / libbpf overview
- Linux sched-ext documentation
- Upstream `scx_simple`
- Aya - Rust library for eBPF programs
- ebpf-go - Go library for eBPF programs.